Enums

Lukáš Hozda

Patterns and error handling

NPRG082 · winter 2026/27

Data modelling

- Structs: several fields present together
- Enums: one of several alternatives, each with its own fields
- Patterns: test the alternative and bind its contents
- Ownership rules apply to the contents too
- Errors and missing values are ordinary cases of this model

Independent fields

```
struct Download {
    finished: bool,
    bytes_received: u64,
    file: Option<String>,
    error: Option<String>,
}
```

- A file and an error can both be present
- A finished download can have neither
- Meaningful combinations depend on conventions outside the type

State-specific data

```
enum Download {
    Receiving { bytes_received: u64 },
    Finished { file: String },
    Failed { error: String },
}
```

- Exactly one variant at a time
- A finished download always includes a file name
- A failed download includes an error instead

Variant construction

```
enum Download {
    Receiving { bytes_received: u64 },
    Finished(String),
    Cancelled,
}
fn main() {
    let state = Download::Receiving { bytes_received: 128 };
    if let Download::Receiving { bytes_received } = state {
        println!("receiving {bytes_received}");
    }
}
```

Product and sum types

- A struct is a **product**: a choice for each field
- An enum is a **sum**: a choice of variant and its payload
- `(bool, u8)`: $2 \times 256 = 512$ possible values
- `enum Level { Off, On(u8) }`: $1 + 256 = 257$
- These counts describe values, not memory size

Enum methods

```
enum Shape { Square(u32), Rectangle(u32, u32) }
impl Shape {
    fn area(&self) -> u32 {
        match self {
            Shape::Square(side) => side * side,
            Shape::Rectangle(w, h) => w * h,
        }
    }
}
fn main() { println!("{}", Shape::Square(3).area()); }
```

Recursive values

```
enum List {
    End,
    Node(i32, List),
}
fn main() {}
```

- E0072: the type would need infinitely large inline storage
- The recursive field needs indirection

Indirection with Box

```
enum List { End, Node(i32, Box<List>) }
fn sum(list: &List) -> i32 {
    match list {
        List::End => 0,
        List::Node(value, tail) => value + sum(tail),
    }
}
fn main() {
    let list = List::Node(7, Box::new(List::End));
    println!("{}", sum(&list));
}
```

Enum representation

- A variant's identity is its **discriminant**
- Storage must accommodate the payload of any variant
- Payload storage can overlap because only one variant is active
- Exact layout is unspecified with Rust's default representation
- Optimizations can encode the discriminant in unused payload values

Optional references

```
use std::mem::size_of;
fn main() {
    assert_eq!(size_of::<Option<&u8>>(), size_of::<&u8>());
    println!("true");
}
```

- A shared reference is non-null; null can encode `None`
- Equal size and alignment are guaranteed for this case
- The same guarantee does not apply to every `Option<T>`

Match expressions

```
fn main() {
    let item = Some(7);
    let description = match item {
        Some(_) => "present",
        None => "absent",
    };
    println!("{description}");
}
```

- First matching arm, one resulting value
- Arm results must have compatible types

Exhaustiveness

```
enum Status { Queued, Running, Finished }
fn main() {
    let state = Status::Queued;
    match state {
        Status::Queued => println!("queued"),
        Status::Running => println!("running"),
    }
}
```

- E0004: `Finished` has no matching arm

Wildcard coverage

- _ matches any value without binding it
- A catch-all arm includes variants added later
- Explicit arms expose missing cases when an enum changes
- Use a catch-all when those other cases genuinely have the same meaning

Binding patterns

```
fn main() {
    let expected = 5;
    match 8 {
        expected => print!("{expected} "),
    }
    println!("{expected}");
}
```

- A binding pattern introduces a name, even if it shadows one
- Comparing with an existing local requires an expression in a guard

Ranges and alternatives

```
fn main() {
    let code = 403;
    let message = match code {
        200..=299 => "success",
        401 | 403 => "denied",
        _ => "other",
    };
    println!("{message}");
}
```

Match guards

```
fn main() {
    let expected = 5;
    match Some(5) {
        Some(value) if value == expected => println!("equal"),
        Some(_) => println!("different"),
        None => println!("absent"),
    }
}
```

- A guard runs after its pattern matches
- A false guard allows matching to continue

Guards and exhaustiveness

```
fn main() {
    let value = 7;
    match value {
        n if n >= 0 => println!("nonnegative"),
        n if n < 0 => println!("negative"),
    }
}
```

- Guard conditions do not establish exhaustive coverage
- Add an unguarded case covering the remaining possibilities

Bindings with @

```
fn main() {
    match 204 {
        code @ 200..=299 => println!("{code}"),
        _ => println!("other"),
    }
}
```

- Bind the matched value and constrain it with another pattern

Struct and tuple patterns

```
struct Point { x: i32, y: i32 }
fn main() {
    let pair = (Point { x: 3, y: 4 }, 9);
    let (Point { x: horizontal, y }, _) = pair;
    println!("{horizontal} {y}");
}
```

- Field renaming, shorthand, and nested patterns

Slice patterns

```
fn ends(values: &[i32]) -> Option<(&i32, &i32)> {
    match values {
        [] | [_] => None,
        [first, .., last] => Some((first, last)),
    }
}
fn main() {
    println!("{:?}", ends(&[2, 4, 5]));
}
```

- `..` ignores the intervening elements, including zero elements

Bound subslices

```
fn main() {
    let values = [1, 2, 3, 4];
    if let [_, middle @ .., _] = values.as_slice() {
        println!("{middle:?}");
    }
}
```

- Here `middle` has type `&[i32]`
- The pattern requires at least two elements

Refutable patterns

```
fn main() {
    let item = Some(7);
    let Some(value) = item;
    println!("{value}");
}
```

- Plain `let` requires an **irrefutable** pattern
- `Some(value)` can fail to match: the type also contains `None`

Conditional binding

```
fn main() {
    let item = Some(7);
    if let Some(value) = item {
        println!("{value}");
    } else {
        println!("absent");
    }
}
```

- `if let`: one pattern of interest, optional fallback

Early exits with let else

```
fn print_value(line: &str) {
    let Some((_, value)) = line.split_once('=') else {
        return;
    };
    println!("{value}");
}
fn main() { print_value("name=oak"); }
```

- A successful binding is available in the surrounding scope
- The `else` branch must diverge: `return`, `break`, `panic!`, …

Repeated matching

```
fn main() {
    let mut stack = vec![1, 2, 3];
    while let Some(value) = stack.pop() {
        print!("{value} ");
    }
}
```

- `pop` removes and returns the last element
- Loop until the pattern fails, here on `None`

Boolean pattern tests

```
fn main() {
    let value = Some(8);
    let positive = matches!(value, Some(n) if n > 0);
    println!("{positive}");
}
```

- `matches!` produces a `bool`
- Pattern bindings are local to the macro's test

Moving a payload

```
fn main() {
    let name = Some(String::from("oak"));
    if let Some(text) = name {
        println!("{text}");
    }
    println!("{name:?}");
}
```

- `text` takes ownership of the inner `String`
- The original option may have been partially moved

Borrowing a payload

```
fn main() {
    let name = Some(String::from("oak"));
    if let Some(text) = &name {
        print!("{text} ");
    }
    println!("{}", name.is_some());
}
```

- Matching through `&name` binds `text: &String`
- Reference binding is inferred from the borrowed value

Mutable pattern bindings

```
fn main() {
    let mut name = Some(String::from("oak"));
    if let Some(text) = &mut name {
        text.push('s');
    }
    println!("{}", name.unwrap());
}
```

- Here `text: &mut String`
- Mutation of the payload without moving it out

Explicit reference bindings

```
fn main() {
    let name = Some(String::from("oak"));
    if let Some(ref text) = name {
        print!("{text} ");
    }
    println!("{}", name.is_some());
}
```

- `ref text` borrows for the binding instead of moving
- `ref` creates a reference; an `&` pattern destructures one

Reference patterns

```
fn main() {
    let point = &(3, 4);
    let &(x, y) = point;
    println!("{x} {y}");
}
```

- The `&` pattern removes a reference layer during matching
- Here the bound coordinates are copied integers, not references

Wildcards and named bindings

```
fn main() {
    let text = String::from("oak");
    let _ = text;
    println!("{text}");
    let _saved = text;
    println!("{text}");
}
```

- _ does not bind or move this string
- `_saved` is an ordinary binding and takes ownership

Borrowed views of Option

```
fn main() {
    let name = Some(String::from("oak"));
    let view: Option<&String> = name.as_ref();
    if let Some(text) = view {
        print!("{} ", text.len());
    }
    println!("{}", name.is_some());
}
```

- `as_ref`: `&Option<T>` to `Option<&T>`

Taking an optional value

```
struct Slot { value: Option<String> }
impl Slot {
    fn take(&mut self) -> Option<String> { self.value.take() }
}
fn main() {
    let mut slot = Slot { value: Some(String::from("oak")) };
    print!("{} ", slot.take().unwrap());
    println!("{}", slot.value.is_none());
}
```

- Move the old value out and leave `None` in its place

Replacing an optional value

```
fn main() {
    let mut name = Some(String::from("oak"));
    let previous = name.replace(String::from("elm"));
    println!("{} {}", previous.unwrap(), name.unwrap());
}
```

- Return the old option and store `Some(new_value)`
- Both `take` and `replace` work through an exclusive borrow

Mapping a value

```
fn main() {
    let name = Some(String::from("oak"));
    let length = name.map(|text| text.len());
    let missing: Option<String> = None;
    println!("{length:?} {:?}", missing.map(|text| text.len()));
}
```

- `|text| text.len()` is a closure: parameter and expression
- `map` consumes the option; its closure runs only for `Some`

Nested and chained operations

```
fn main() {
    let value = Some(200_u8);
    let nested = value.map(|n| n.checked_mul(2));
    let flat = value.and_then(|n| n.checked_mul(2));
    println!("{nested:?} {flat:?}");
}
```

- `map`: wrap the result, here `Option<Option<u8>>`
- `and_then`: the next operation already returns an `Option`

Eager and lazy defaults

```
fn fallback() -> u32 { print!("computed "); 0 }
fn main() {
    let present = Some(7);
    let eager = present.unwrap_or(fallback());
    let lazy = present.unwrap_or_else(|| fallback());
    println!("{eager} {lazy}");
}
```

- Ordinary arguments are evaluated before the method call
- Pass a closure to compute a fallback only when needed

Absence and failure

- `Option<T>`: a value may be absent in normal operation
- `Result<T, E>`: an operation may fail with a described cause
- `Result<Option<T>, E>`: success can itself contain no value
- Example: missing configuration is allowed; malformed configuration is an error

Result contracts

```
fn find_user(id: u64) -> Option<User>
fn parse_user(text: &str) -> Result<User, ParseError>
fn load_user(id: u64) -> Result<Option<User>, StorageError>
```

- The return type distinguishes these outcomes for the caller
- Error values can carry structured information
- `Err` is a returned value, not an exception in flight

Error propagation

```
use std::num::ParseIntError;
fn is_even(text: &str) -> Result<bool, ParseIntError> {
    let number = text.parse::<u32>()?;
    Ok(number % 2 == 0)
}
fn main() { println!("{:?}", is_even("42")); }
```

- On `Ok`, unwrap the success value
- On `Err`, return early from the enclosing function

Propagation as a match

```
let number = match text.parse::<u32>() {
    Ok(value) => value,
    Err(error) => return Err(error.into()),
};
```

- The `Result` case of `?`, written explicitly
- The enclosing function needs a compatible result type
- Error conversion uses `From`; trait implementations come later

Propagation boundaries

```
fn main() {
    let number = "42".parse::<u32>()?;
    println!("{number}");
}
```

- `main` returns `()` here, so `?` cannot return a `Result`
- `?` returns from this function, not just the nearest block

Optional propagation

```
fn initials(first: &str, last: &str) -> Option<(char, char)> {
    let first = first.chars().next()?;
    let last = last.chars().next()?;
    Some((first, last))
}
fn main() { println!("{:?}", initials("Ada", "Lovelace")); }
```

- With `Option`, `?` returns `None` early
- It does not automatically translate absence into an error

Absence as an error

```
#[derive(Debug)]
enum ConfigError { Missing }
fn main() {
    let setting: Option<u16> = None;
    let result = setting.ok_or(ConfigError::Missing);
    println!("{result:?}");
}
```

- `ok_or`: `Option<T>` to `Result<T, E>`
- `ok_or_else`: construct the error only for `None`

Domain error types

```
use std::num::ParseIntError;

#[derive(Debug)]
enum PortError {
    Parse(ParseIntError),
    Zero,
}
```

- Preserve the parsing error as a payload
- Distinguish malformed input from a domain rule

Error conversion and validation

```
use std::num::ParseIntError;
#[derive(Debug)]
enum PortError { Parse(ParseIntError), Zero }
fn parse_port(text: &str) -> Result<u16, PortError> {
    let port = text.parse::<u16>().map_err(PortError::Parse)?;
    if port == 0 { return Err(PortError::Zero); }
    Ok(port)
}
fn main() { println!("{:?}", parse_port("0")); }
```

- `map_err` transforms the error, leaving success unchanged

Validation and types

```
use std::num::NonZeroU16;
fn main() {
    let rejected = NonZeroU16::new(0);
    let accepted = NonZeroU16::new(443);
    println!("{} {}", rejected.is_some(), accepted.is_some());
}
```

- After construction, a `NonZeroU16` always contains a nonzero value
- Return validated types when later code depends on that invariant

Error information

- Error variants identify cases callers can handle differently
- Payloads retain relevant input, locations, and underlying causes
- `Debug` (`{:?}`): programmer-facing representation
- `Display` (`{}`): human-readable formatting, when implemented
- `std::error::Error`: a common interface, including an optional source

Handling a specific failure

```
use std::num::IntErrorKind::InvalidDigit;
fn main() {
    match "oak".parse::<u16>() {
        Ok(port) => println!("{port}"),
        Err(error) => match error.kind() {
            InvalidDigit => println!("not an integer"),
            _ => println!("{error}"),
        },
    }
}
```

Error-handling boundaries

- Low-level operation: return the cause to the caller
- Caller: recover, add context, or propagate
- Application boundary: report the error and choose an exit status
- Logging at every propagation step produces duplicate reports
- Replace a failure with a default only when that default is meaningful

Fallible main

```
use std::num::ParseIntError;
fn main() -> Result<(), ParseIntError> {
    let value = "42".parse::<u32>()?;
    println!("{value}");
    Ok(())
}
```

- `Ok(())` indicates successful completion
- Returning `Err` prints its `Debug` representation and exits unsuccessfully

Optional parsing

```
use std::num::ParseIntError;
fn setting(
    text: Option<&str>,
) -> Result<Option<u16>, ParseIntError> {
    text.map(|s| s.parse::<u16>()).transpose()
}
fn main() {
    println!("{:?} {:?}", setting(Some("42")), setting(None));
}
```

Transposing optional results

```
match optional_result {
    Some(Ok(value)) => Ok(Some(value)),
    Some(Err(error)) => Err(error),
    None => Ok(None),
}
```

- `transpose`: `Option<Result<T, E>>` to `Result<Option<T>, E>`
- Missing input is successful absence; malformed input is an error

Information loss

```
fn main() {
    let malformed = "oak".parse::<u16>().ok();
    let out_of_range = "70000".parse::<u16>().ok();
    println!("{malformed:?} {out_of_range:?}");
}
```

- `ok()` discards the error value
- Both different failures become the same absence
- Useful when the distinction is irrelevant to the caller

Panics

- A panic signals failure to continue under the current assumptions
- Examples: an assertion failure or indexing outside a slice
- `unwrap` and `expect` panic on `None` or `Err`
- Expected input failures belong in `Option` or `Result`
- Panic handling is separate from matching a returned error

Unwrapping an invariant

```
fn main() {
    let words = ["oak", "elm"];
    let first = words.first().expect("words is nonempty");
    println!("{first}");
}
```

- State the reason success is expected in the message
- Keep the invariant visible near the unwrap
- Use a fallible return type when failure is part of the API

Cleanup paths

- Ordinary return, including `?`: drop initialized local values
- With **unwinding**, a panic also drops locals on the unwound stack
- With **abort**, the process terminates without unwinding
- `catch_unwind` catches unwinding panics, not returned `Err` values
- Catching a panic does not undo earlier mutations or external effects

Data and control flow

- Represent alternative states with enum variants and their payloads
- Use patterns to inspect, borrow, mutate, or move those payloads
- Keep absence, malformed input, and domain errors distinct
- Propagate errors until a caller has a useful recovery or reporting policy
- Make assumptions explicit at the point where a panic is acceptable